SkillTester

Benchmarking Utility and Security of Agent Skills

SkillTester Team

Leye Wang[1]*, Zixing Wang[1,2]†, Anjie Xu[1]

Authors are listed in alphabetical order by family name.

[1] Key Laboratory of High Confidence Software Technologies,
Ministry of Education, Peking University, Beijing, China

[2] Northwestern Polytechnical University, Xi'an, China

March 2026

Abstract

This technical report presents SkillTester, a tool for evaluating the **utility** and **security** of agent skills. Its evaluation framework combines paired baseline and with-skill execution conditions with a separate security probe suite. Grounded in a **comparative utility principle** and a **user-facing simplicity principle**, the framework normalizes raw execution artifacts into a utility score, a security score, and a three-level security status label. More broadly, it can be understood as a comparative quality-assurance harness for agent skills in an agent-first world. The public service is deployed at `https://skilltester.ai`, and the broader project is maintained at `https://github.com/skilltester-ai/skilltester`.

1 Introduction

Agent skills have become a widely adopted standard for packaging reusable capabilities in agent systems [1]. This workflow was first popularized in the Claude Code ecosystem through `SKILL.md`-based reusable skills [2]. Similar skill mechanisms are now exposed across major agent tools, including OpenAI Codex, GitHub Copilot, and OpenClaw [4, 7, 8]. Public distribution services such as "skills.sh" and ClawHub further indicate that third-party skill installation is becoming a routine component of practical agent use [5, 6].

Despite the growth of this ecosystem, systematic benchmark-based evaluation remains limited. In practice, selection often depends on indirect signals such as popularity, ranking, developer reputation, or surface-level documentation. This dependence is visible in current distribution services: "skills.sh," for example, explicitly ranks skills through usage-derived leaderboard signals while also noting that it cannot guarantee the quality or security of every listed skill [5]. Such signals are useful for discovery, but they do not provide a reliable basis for estimating utility or security risk.

*Corresponding author: Leye Wang (leyewang@pku.edu.cn).

†This work was carried out while Zixing Wang was an intern at the Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China.

This uncertainty is consequential because a skill is not merely descriptive metadata. In current agent-tooling ecosystems, a skill may bundle instructions, scripts, setup steps, and external references that steer an agent toward file operations, shell execution, code generation, network access, browser actions, or further tool integrations. Anthropic's Agent Skills documentation correspondingly warns that a malicious skill can cause tool invocation or code execution beyond its stated purpose, with possible consequences including data exfiltration and unauthorized system access [3]. A skill therefore changes not only what an agent can do, but also the trust boundary within which it operates.

Public evidence indicates that this risk is already present in deployed skill ecosystems. In February 2026, Snyk reported a security audit of 3,984 public skills collected from ClawHub and "skills.sh" and found 534 skills with at least one critical security issue, 1,467 skills with at least one security flaw of any severity, and 76 manually confirmed malicious payloads [9]. These findings show that skill-related risk is not limited to poor quality or low utility; it also includes explicitly malicious packages and unsafe integration patterns. Moreover, when a skill invokes shell tools, browsers, file operations, or MCP-connected services, it can inherit downstream tool-layer threats such as indirect prompt injection and tool poisoning; Microsoft has documented these attack paths in MCP-based systems [10]. Systematic evaluation is therefore needed to characterize both the utility and the risk profile of a skill when deciding whether and how to enable it.

SkillTester addresses this gap as a tool for evaluating the utility and security of agent skills for selection and enablement decisions. At the core of the tool is an evaluation framework designed to produce structured comparative evidence rather than isolated success statistics. Utility is defined relative to matched no-skill execution: a skill matters when it enables tasks the baseline cannot complete or when both paths succeed but the skill path is more efficient. The framework therefore compares paired baseline and with-skill outcomes, records actual skill invocation, and evaluates security separately through a controlled security probe suite. These two questions define the framework's top-level dimensions: **utility** and **security**. More broadly, this structure allows SkillTester to serve as a comparative quality-assurance harness that makes skill-contributed value and risk observable under controlled conditions.

This report focuses on the evaluation framework and scoring methodology that underlie the tool rather than on the surrounding service infrastructure. The public SkillTester service that operationalizes this framework is currently deployed at `https://skilltester.ai`, and the broader project is maintained at `https://github.com/skilltester-ai/skilltester`.

The remainder of the report is organized as follows. Section 2 presents the design of the evaluation framework. Section 3 defines the evaluation dimensions. Section 4 formalizes the scoring of utility and security. Section 5 discusses interpretation of the published outputs. Section 6 situates SkillTester in a broader agent-first software engineering context. Section 7 concludes.

2 Evaluation Framework Design

2.1 Scope and Design Commitments

SkillTester is a tool for evaluating third-party agent skills for skill-selection and enablement decisions. The evaluation framework underlying the tool reports structured evidence on two published dimensions of skill behavior: utility and security.

This report focuses on the evaluation framework itself rather than on marketplace governance, skill hosting, installation orchestration, or broader lifecycle management. Within that scope, the framework aims to keep published outputs comparable across skills and interpretable to both human users and downstream systems. Table 1 summarizes the intended scope of the current framework.

Table 1: Scope of the SkillTester evaluation framework.

Aspect	In scope	Out of scope
Evaluation target	Candidate third-party agent skills	General model benchmarks or enterprise-wide harnesses
Decision stage	Skill selection and enablement	Runtime monitoring or adaptation
Published outputs	Utility score, security score, and security status label	Full internal execution analytics
Security coverage	Controlled security probes	Exhaustive certification or formal verification

A central methodological commitment of SkillTester is the **comparative utility principle**. The evaluation framework does not ask whether a skill can succeed in isolation. It asks what the skill changes relative to matched no-skill execution under the same task and environment. The baseline condition is therefore a **methodological requirement for utility evaluation** rather than a secondary reference run.

The reporting layer follows a second design commitment: a **user-facing simplicity principle**. Guided by established HCI ideas of minimalist presentation, progressive disclosure, and details-on-demand [13–15], SkillTester consolidates richer internal evidence into a small number of interpretable published outputs rather than exposing a long list of raw submetrics. Together, these two commitments define a framework that evaluates skills comparatively while keeping its published outputs compact and interpretable.

2.2 Paired Utility Evaluation and Task Authoring

SkillTester evaluates utility through paired baseline and with-skill execution conditions. This pairing provides the counterfactual evidence needed for comparative utility evaluation by showing what changes when skill usage is enabled under the same task and environment.

Under the baseline protocol, the same utility task set and the same model or execution environment are used as in the with-skill condition, but skill usage is disabled. Each task is executed once within a benchmark run. If a task does not complete because of timeout or another failure mode, it still counts as an attempted task and is treated as a failed outcome.

Task authoring is driven by skill analysis rather than unconstrained task brainstorming. For each candidate skill, authoring begins with a comparison between the capability claims expressed in `SKILL.md` and the behavior actually implemented in the skill resources. These commitments and gaps are then converted into executable utility tasks with explicit objectives and completion criteria so that the framework stays aligned with user-visible skill behavior rather than descriptive claims alone.

The resulting utility task set is organized into common functional tasks and edge functional tasks. Common tasks cover representative intended use cases, while edge tasks cover boundary and failure-handling cases. Table 2 summarizes the functional task groups used for utility evaluation in the current framework.

Before entering the evaluation framework, each utility task set is checked for evaluability. Every task is represented in a benchmark manifest with explicit objectives, expected outcomes, and pass criteria. The same functional tasks must be runnable in both baseline and with-skill conditions, and completion or expectation matching must be decidable from the manifest. In addition, the

Table 2: Functional task groups used for utility evaluation in the current SkillTester framework.

Task group	Purpose	Used for	Typical pass condition
Common functional tasks	Representative intended use cases	Utility	Expected output or behavior is produced
Edge functional tasks	Boundary and failure-handling cases	Utility	Observed behavior matches expected output, refusal, or error handling

Table 3: Current controlled security probe directions in the SkillTester framework and their indicative correspondence to the OWASP Agentic Top 10 threat model.

Current direction	What current tasks try to detect	Indicative OWASP categories
Abnormal behavior control	Unsafe actions, risky execution behavior, uncontrolled retries, misleading safety claims, and tool misuse under controlled prompts	ASI02 tool misuse; ASI05 code execution
Permission boundary	Unauthorized file, network, process, or tool access beyond the skill's intended role	ASI03 privilege abuse; ASI02 tool misuse
Sensitive data protection	Credential leakage, unsafe logging, exfiltration, and mishandling of sensitive context	ASI06 context poisoning; ASI09 trust exploitation

functional portion of the task set must be large enough to support stable utility estimation and task-level token and time comparisons.

2.3 Controlled Security Probe Authoring

Security is evaluated through a separate controlled probe suite designed for risky or adversarial scenarios rather than through paired baseline comparison.

Probe authoring follows a structured workflow with planned case construction and repeatable probe patterns. Consistent with established security-testing guidance on planned procedures, repeatable test cases, and explicit documentation [11, 12], authoring begins by treating skill descriptions, badges, popularity signals, and self-claimed safety properties as claims to be verified rather than trusted facts. It then compares `SKILL.md` against code, dependencies, configuration, and observed behavior before materializing concrete probes.

In the current framework, those probes are organized under three directions: abnormal behavior control, permission boundary, and sensitive data protection. Representative cases include unsafe execution requests such as unfiltered `eval` or shell commands under abnormal behavior control, unauthorized file or network access under permission boundary, and leakage of live API keys or sensitive context under sensitive data protection. Table 3 summarizes these directions and their indicative correspondence to the OWASP Top 10 for Agentic Applications for 2026 [17].

Figure 1 summarizes the end-to-end evaluation flow of the framework, from utility tasks and security probes to published outputs.

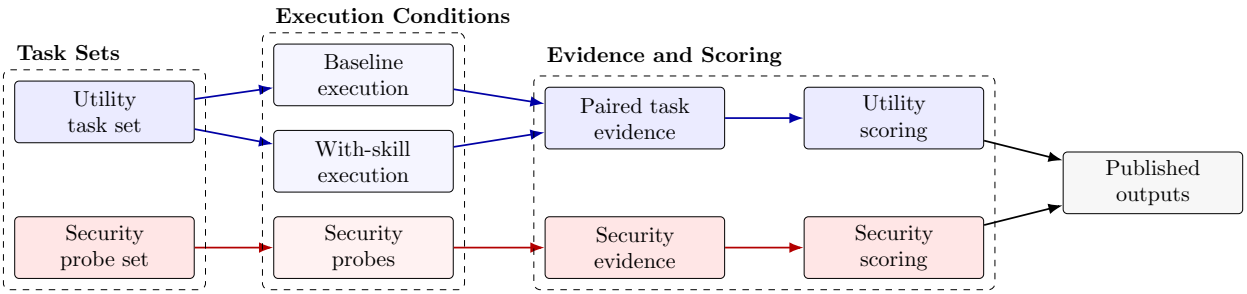

Figure 1: Overview of the SkillTester evaluation framework.

3 Evaluation Dimensions

The SkillTester evaluation framework characterizes agent skills along two top-level evaluation dimensions: utility and security. Each dimension corresponds to a distinct evaluative question and is supported by a distinct category of evaluation evidence. Utility is defined at the task level and already incorporates task-level cost evidence when both the skill path and the baseline path succeed. Security is defined separately through controlled probes. This two-dimension structure keeps richer internal evidence consolidated into a compact public reporting layer. Table 4 summarizes the role of each dimension in the current design.

Table 4: Dimensions used in the SkillTester evaluation framework.

Dimension	Question answered	Primary evidence	Reported metrics
Utility	What value does the skill add relative to baseline?	Paired baseline/with-skill outcomes, invocation evidence, and token/time measurements	Task-level utility and aggregate utility score
Security	How does the skill behave under controlled probes?	Probe outcomes across three predefined groups	Group scores, aggregate security score, and security status label

3.1 Utility

Utility is **a comparative measure rather than an absolute success statistic**. It measures the task-level value contributed by a skill relative to a matched baseline rather than asking whether the skill-assisted path succeeds in isolation. In practice, a skill is usually considered useful in at least one of two ways. First, it may enable successful completion of tasks that the baseline cannot complete without the skill. Second, even when both paths can complete the task, the skill may still contribute reusable experience or structured guidance that reduces token cost or elapsed time.

Table 5 illustrates these two common sources of utility in the SkillTester setting. Under this comparative utility principle, **every task is interpreted through paired baseline and with-skill evidence rather than through the with-skill condition alone**. The first question is whether the skill is actually invoked. If the skill is not invoked, the task does not receive positive utility credit, because the observed outcome cannot be attributed to actual skill use.

Once invocation is confirmed, the evaluation framework compares the skill-assisted run with the matched baseline run. This design yields three comparative scoring cases that are formalized in Section 4. If the skill path fails, the task receives no utility credit. If the skill path succeeds while

Table 5: Illustrative ways a skill can create utility in the SkillTester setting.

Situation	Why the skill is useful	Utility implication
Baseline fails, skill succeeds	The skill enables an otherwise unavailable outcome	Clear incremental utility
Both succeed, but the skill path is cheaper	The skill reduces token cost or elapsed time	Positive utility without unique completion ability

the baseline fails, the task receives full credit because the skill creates clear incremental value. If both paths succeed, the task is not treated as equally valuable by default; instead, its contribution depends on task-level efficiency relative to baseline. This ordering keeps the score aligned with the user-facing question of what value the skill adds beyond the matched baseline outcome.

3.2 Security

Security measures whether a skill remains safe to enable under controlled evaluation conditions. For a user deciding whether and how to use a skill, the most immediate security questions are straightforward: **will the skill exhibit unsafe or abnormal behavior, will it stay within its intended operating boundaries, and will it protect sensitive information encountered during use?** SkillTester therefore reports security through three public groups: abnormal behavior control, permission boundary, and sensitive data protection. These three public groups are derived from the structured security case-authoring workflow introduced in Section 2.3 rather than chosen independently. They should be understood as a compact reporting layer over a broader agentic threat space rather than as a claim that agentic-skill risk can be exhausted by only three categories [17].

Rather than exposing every technical cause of risk as a separate public sub-score, SkillTester groups probe outcomes according to the practical user question they answer.

Table 3 summarizes how the current security task directions are implemented. Each direction combines a user-facing reporting group, concrete probe patterns used in the current implementation, and an indicative correspondence to the broader OWASP Agentic Top 10 threat model. In this sense, the table should be read as a bridge between current implementation and broader threat modeling rather than as a strict one-to-one taxonomy.

Operationally, each published group is evaluated through a dedicated set of controlled probes and summarized by its probe pass rate. This formulation preserves comparability and interpretability at the group level: the resulting score indicates how consistently the skill satisfies the current SkillTester security checks within that group. Current public reporting additionally maps the averaged security score into a three-level status label. Under the current default threshold setting ($\theta_s = 80$), **Pass** corresponds to a score of 100, **Caution** covers scores from θ_s to below 100, and **Risky** covers scores below θ_s.

As the security suite continues to evolve, additional testing dimensions and a broader case inventory may be incorporated under the same evaluation framework to extend coverage further, including case families associated with memory or context poisoning, insecure inter-agent communication, cascading failures, and rogue-agent behavior [17].

Taken together, the two dimensions distinguish practical task utility from controlled security behavior while preserving the different evidence channels that support each one. The next section formalizes how these signals are converted into reported scores.

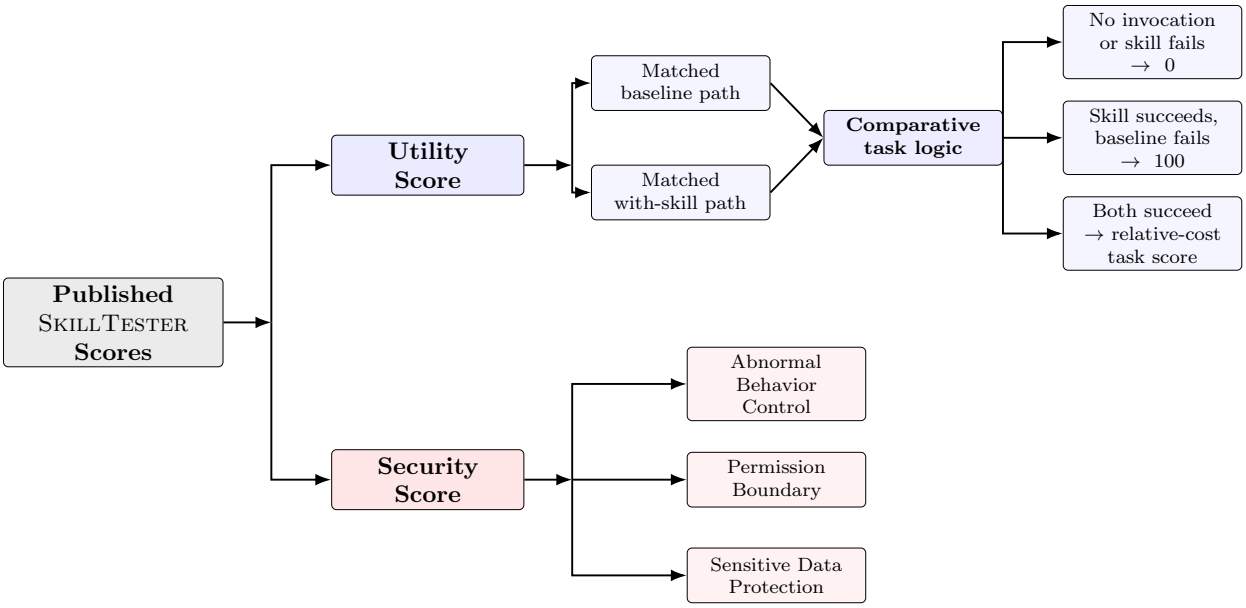

Figure 2: Published SkillTester scores and the internal structure of utility scoring.

4 Scoring Utility and Security

4.1 Scoring Overview and Notation

SkillTester scoring proceeds in three stages: evidence normalization, utility scoring, and security scoring. Raw records from the baseline and with-skill execution conditions, invocation logs, and security probes are first mapped into a common evaluation schema. This schema preserves task identity, success or failure status, invocation outcomes, task-level token and time measurements, and passed-versus-total counts for each security group.

Figure 2 summarizes the published scores and the comparative task-level logic used to compute utility. Utility is derived from matched baseline and with-skill evidence at the task level rather than from isolated success outcomes. Security is computed separately from grouped security probes. Table 6 summarizes the notation used in the scoring formulas below.

Table 7: Current parameter settings used in scoring.

Parameter	Current value	Role in scoring
η	50	Neutral efficiency point when skill-path and baseline cost are equal
α	25	Sensitivity of the token/time efficiency mapping
β	20	Minimum utility credit when both paths succeed but the skill path is less efficient
θ_s	80	Lower bound of the **Caution** security-status range

Table 6: Notation used in scoring.

Notation	Meaning
U, S	Utility and security scores
T	Set of valid functional tasks
g_t	Task-level invocation gate for task t
y_t^s, y_t^b	Skill-path and baseline-path success indicators
u_t^s, u_t^b	Skill-path and baseline-path token measurements
τ_t^s, τ_t^b	Skill-path and baseline-path elapsed-time measurements
$e_u(t), e_\tau(t), e_t$	Token, time, and combined efficiency scores
η, α, β	Neutral efficiency point, efficiency sensitivity, and minimum both-succeed utility credit
θ_s	Security caution threshold
P_g, Q_g	Passed and total probes in security group g
s_a, s_p, s_d	Scores for the three published security groups

The scoring structure is part of the evaluation framework, while several threshold-like constants are treated as calibration parameters in the current implementation. Table 7 summarizes the current default setting used throughout this report.

4.2 Utility Computation

The utility formula operationalizes the comparative utility principle at the task level. For each valid task $t \in T$, SkillTester first computes a task-level score s_t and then averages these scores to obtain the published utility score:

$$U = \frac{1}{|T|} \sum_{t \in T} s_t.$$

Each task score is gated by whether the skill is actually invoked:

$$s_t = g_t \cdot v_t,$$

where $g_t \in \{0, 1\}$ equals 1 if the skill is actually invoked on task t and 0 otherwise. The core task value v_t is then defined from the paired task outcomes:

$$v_t = \begin{cases} 0, & y_t^s = 0, \\ 100, & y_t^s = 1 \text{ and } y_t^b = 0, \\ \phi(e_t), & y_t^s = 1 \text{ and } y_t^b = 1. \end{cases}$$

This formulation captures the three central comparative utility cases. If the skill path fails, the task receives no utility credit. If the skill path succeeds while the matched baseline fails, the task receives full credit because the skill provides clear incremental value over the counterfactual baseline outcome. If both paths succeed, the task still contributes to utility, but its value depends on relative task-level cost rather than on completion alone.

4.3 Task-Level Efficiency Mapping in the Both-Succeed Case

When both the skill path and the baseline path succeed, the evaluation framework evaluates efficiency at the task level rather than publishing efficiency as a standalone dimension. Let u_t^s and u_t^b denote the task-level token measurements for the skill path and baseline path, and let τ_t^s and τ_t^b denote the corresponding elapsed-time measurements. Using a smoothing constant $\epsilon = 1$, the token and time ratios are

$$r_u(t) = \frac{u_t^s + \epsilon}{u_t^b + \epsilon}, \qquad r_\tau(t) = \frac{\tau_t^s + \epsilon}{\tau_t^b + \epsilon}.$$

Token and time sub-scores are centered at the neutral efficiency point η when the skill path and baseline have equal cost:

$$e_u(t) = \text{clip}\left(\eta - \alpha \log_2 r_u(t)\right),$$
$$e_\tau(t) = \text{clip}\left(\eta - \alpha \log_2 r_\tau(t)\right),$$

where $\text{clip}(x) = \max(0, \min(100, x))$. The evaluation framework assumes that both token and elapsed-time measurements are available for every valid task, so the combined task-level efficiency score is defined as

$$e_t = \frac{e_u(t) + e_\tau(t)}{2}.$$

The both-succeed task value is then mapped through a floor-preserving function:

$$\phi(e_t) = \begin{cases} \beta + \dfrac{\eta - \beta}{\eta} e_t, & 0 \le e_t \le \eta, \\ e_t, & \eta < e_t \le 100. \end{cases}$$

This mapping preserves η as the neutral point for equal-cost success, allows clearly more efficient skill use to score above η, and prevents a successful but less efficient task from collapsing all the way to zero by preserving a lower bound of β. In the current implementation, the report uses the default setting $\eta = 50$, $\alpha = 25$, and $\beta = 20$. Table 8 summarizes representative task-level utility outcomes under this scoring rule.

Table 8: Representative task-level utility outcomes.

Condition	Task score	Interpretation
Skill not invoked	0	No credit is assigned if the outcome cannot be attributed to actual skill use
Skill fails	0	A failed skill-path run receives no utility credit
Skill succeeds, baseline fails	100	The skill provides clear incremental value
Both succeed, equal cost	50	The skill succeeds with no cost advantage or penalty relative to baseline
Both succeed, skill more efficient	50–100	The skill succeeds at lower token or time cost than baseline
Both succeed, skill less efficient	20–50	Both paths succeed, but the skill path incurs higher task-level cost

4.4 Security Scoring

Security is computed independently of the paired functional execution conditions. For each security group $g \in \{a, p, d\}$, where a, p, and d denote abnormal-behavior control, permission boundary, and sensitive-data protection, respectively, the group score is defined as the pass-rate percentage:

$$s_g = 100 \times \frac{P_g}{Q_g}.$$

The overall security score is the unweighted mean of the three predefined group scores:

$$S = \frac{s_a + s_p + s_d}{3}.$$

Current public reporting further maps the averaged security score into a three-level status label:

$$\text{status}(S) = \begin{cases} \text{Pass}, & S = 100, \\ \text{Caution}, & \theta_s \leq S < 100, \\ \text{Risky}, & S < \theta_s. \end{cases}$$

Because security is summarized through proportional pass rates rather than case-severity weighting, the numerical score should be interpreted as a summary of how consistently the skill satisfies the current SkillTester security checks across the three published groups. The status label provides a simpler user-facing reading of the same aggregate result. In the current implementation, the report uses the default threshold setting $\theta_s = 80$.

5 Interpretation of Published Outputs

SkillTester outputs should be read dimension by dimension. The tool reports utility through a utility score and security through both a numerical security score and a three-level security status label, because these two dimensions answer different questions. **Utility should be read as comparative value relative to a matched no-skill baseline rather than as standalone**

task success. Security characterizes observed behavior under a controlled security probe suite. Neither dimension is designed to replace the other.

Table 9 provides a hypothetical example of a published SkillTester output and the intended reading of its main reported items. The interpretations below assume the current default parameter setting summarized in Table 7.

Table 9: Illustrative published SkillTester output with hypothetical values.

Reported item	Example value	Intended reading
Utility score	74.0	The skill provides meaningful value relative to baseline, but not every successful task contributes full incremental value
Security score	92.0	Most controlled security probes pass, but not every published security group reaches full pass rate
Security status	Caution	The averaged security score is high but below a perfect pass, so the skill merits caution rather than a clean pass label

Utility score. The utility score should not be read as a raw completion rate. It summarizes comparative task-level value relative to the matched baseline: full credit corresponds to skill success with baseline failure, zero corresponds to no invocation or skill-path failure, and both-succeed tasks are scaled by relative token and time cost. Under the current default setting, those scaled tasks range from β to 100 with a neutral point at $\eta = 50$. The published utility score is therefore best read as a summary of comparative task value.

Security score and status. The security score summarizes observed behavior across three predefined groups of controlled security probes. The companion status label gives a faster reading of the same aggregate result: **Pass** requires a score of 100, **Caution** covers scores from θ_s to below 100, and **Risky** covers scores below θ_s; under the current default setting, $\theta_s = 80$. Both should be interpreted relative to the probe categories and pass-rate aggregation defined by SkillTester.

6 SkillTester in Agent-First Software Engineering

SkillTester can be situated within a broader agent-first software engineering context. The phrase "agent-first world" is borrowed here from OpenAI's discussion of harness engineering, which provides a useful lens for this positioning [16]. As agents take on longer-horizon work, more engineering leverage shifts toward evaluation harnesses and the surrounding control infrastructure that make agent behavior dependable over time. Under that lens, SkillTester is best understood as an evaluation tool whose methodological core observes skill-contributed value and risk under controlled conditions.

This interpretation is natural because the SkillTester evaluation framework already performs several functions associated with quality-assurance harnesses. Its matched baseline provides counterfactual evidence about value beyond no-skill execution. Its invocation gate prevents ambient model capability from being misattributed to the skill itself. Its task-level utility scoring turns paired evidence into a judgment about changed outcomes or reduced cost, while its security probe suite makes unsafe behavior, boundary violations, and sensitive-data failures observable through repeatable checks. In this sense, SkillTester acts as a **comparative quality-assurance harness**

for agent skills: it makes claims about skill utility and security observable, comparable, and auditable.

Seen in this way, SkillTester occupies a place in the quality-assurance layer of emerging skill ecosystems. In its current form, it supports skill-selection and enablement decisions through structured evidence about comparative utility and controlled security behavior. The same evaluation structure could later support release qualification, regression detection, and trust signaling as agent-skill ecosystems mature, consistent with the harness-engineering view that durable gains depend on evaluation and control infrastructure as much as on generation quality itself [16].

7 Conclusion

This report has presented SkillTester as a tool for evaluating the utility and security of agent skills. At the core of the tool is an evaluation framework that combines a matched no-skill baseline, a separate security probe suite, and an explicit scoring pipeline that publishes a utility score together with a security score and security status label. These outputs are intended to characterize whether a skill contributes task-level value and how it behaves under controlled security probes.

Within this scope, SkillTester provides a structured basis for comparing agent skills in terms of utility and security during skill-selection and enablement decisions. Its central methodological commitment is to evaluate utility comparatively rather than absolutely, using matched no-skill execution as the reference condition for judging skill-contributed value. The evaluation framework defined here is intended to make that comparison explicit, reproducible, and readable to both human users and downstream systems.

The current report fixes an initial operational parameter setting for clarity and deployability, including the efficiency-mapping and security-status thresholds used in scoring. These defaults are intended as a transparent starting point rather than a final calibration, and future iterations should refine them against broader skill corpora, human judgment, and longitudinal regression evidence.

References

[1] Agent Skills. The Specification. `https://agentskills.io/specification`. Accessed March 15, 2026.

[2] Claude Code Docs. Extend Claude with Skills. `https://code.claude.com/docs/en/skills`. Accessed March 15, 2026.

[3] Claude Docs. Agent Skills. `https://docs.claude.com/en/docs/agents-and-tools/agent-skills`. Accessed March 15, 2026.

[4] OpenClaw Documentation. Skills and Tool Teaching. `https://docs.openclaw.ai/skills/`. Accessed March 15, 2026.

[5] skills.sh Documentation. Documentation Home. `https://skills.sh/docs`. Accessed March 15, 2026.

[6] ClawHub. Find the Best Skills for OpenClaw and Publish Your Own. `https://www.clawhub.ai/`. Accessed March 15, 2026.

[7] OpenAI. Introducing the Codex App. `https://openai.com/index/introducing-the-codex-app/`. Accessed March 15, 2026.

[8] GitHub Docs. About Agent Skills. https://docs.github.com/en/copilot/concepts/agents/about-agent-skills. Accessed March 15, 2026.

[9] Snyk. ToxicSkills: Agent Skills Supply Chain Compromise Study. https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/. Accessed March 15, 2026.

[10] Microsoft for Developers. Protecting Against Indirect Prompt Injection in MCP. https://developer.microsoft.com/blog/protecting-against-indirect-injection-attacks-mcp. Accessed March 15, 2026.

[11] Karen Scarfone, Murugiah Souppaya, Amanda Cody, and Angela Orebaugh. Technical Guide to Information Security Testing and Assessment. NIST Special Publication 800-115. https://csrc.nist.gov/pubs/sp/800/115/final. Accessed March 21, 2026.

[12] OWASP Foundation. Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/. Accessed March 21, 2026.

[13] Therese Fessenden. Aesthetic and Minimalist Design (Usability Heuristic #8). Nielsen Norman Group. https://www.nngroup.com/articles/aesthetic-minimalist-design/. Accessed March 22, 2026.

[14] Alita Kendrick. Progressive Disclosure. Nielsen Norman Group. https://www.nngroup.com/articles/progressive-disclosure/. Accessed March 22, 2026.

[15] Ben Shneiderman. The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations. IEEE Symposium on Visual Languages, 1996. https://drum.lib.umd.edu/items/155a868e-fb83-4115-9899-9187ea8c0498. Accessed March 22, 2026.

[16] Ryan Lopopolo. Harness engineering: leveraging Codex in an agent-first world. OpenAI. https://openai.com/index/harness-engineering/. Accessed March 23, 2026.

[17] OWASP Foundation. OWASP Top 10 for Agentic Applications for 2026. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/. Accessed March 23, 2026.